

Documento de Requisitos del Producto (PRD)

E-COMMERCE MVP & ESTRATEGIA DE DESPLIEGUE MANUAL

Proyecto: E-Commerce MVP (Portafolio)

Autor: Ana (Data Analyst / Developer)

Framework: Django & Bootstrap

Base de Datos: SQLite3

1. Propósito y Objetivos

El propósito de este proyecto es consolidar y entregar la versión final de una aplicación web de comercio electrónico (E-Commerce). El producto terminado actuará como una pieza fundamental de portafolio profesional, demostrando la capacidad técnica para integrar de forma estable la lógica de negocio backend, el manejo de persistencia mediante ORM, interfaces dinámicas y un flujo completo de despliegue en producción.

Objetivos Clave

- Consolidar el flujo principal de negocio de forma óptima y libre de errores: *Catálogo* → *Carrito* → *Confirmación de Compra*.
- Garantizar la persistencia integral de los datos en el entorno productivo utilizando exclusivamente `SQLite3`.
- Implementar un proceso de despliegue manual exitoso y reproducible en la plataforma **Render**, prescindiendo por completo del archivo estructurado `render.yaml`.
- Asegurar un repositorio en GitHub limpio, con un código bien estructurado y documentación transparente lista para auditoría de reclutadores.

2. Roles de Usuario (User Personas)

- **Cliente (Usuario Final Autenticado):** Rol encargado de consumir el catálogo público, operar las cantidades dentro de su carrito de compras y procesar la confirmación del pedido, lo cual requiere de una sesión activa.
- **Administrador (Staff):** Rol técnico de gestión interna que cuenta con acceso total y privilegios exclusivos sobre el catálogo de la tienda para crear, modificar, auditar y eliminar registros.

3. Alcance del MVP y Requisitos Funcionales

3.1 Autenticación y Control de Acceso

- **RF-1.1:** El sistema debe proveer de manera nativa los flujos y vistas indispensables para el inicio (Login) y cierre de sesión (Logout) de cuentas.

- **RF-1.2:** Los visitantes anónimos o invitados podrán examinar el catálogo y detalles, pero es mandatorio que se autenticquen en la plataforma para añadir ítems al carrito, alterar cantidades y efectuar la orden de compra.
- **RF-1.3:** Restricción estricta de rutas: El panel y los endpoints encargados del CRUD de productos deben estar protegidos por middleware o decoradores, garantizando el acceso único a usuarios administradores (`is_staff = True`).

3.2 Catálogo y Persistencia (ORM)

- **RF-2.1:** Los datos de los artículos comerciales deben consultarse y renderizarse de forma completamente dinámica en el frontend mediante el ORM nativo de Django.
- **RF-2.2:** El Administrador debe disponer de un panel web accesible para realizar operaciones CRUD completas sobre los registros de productos.
- **RF-2.3:** Cada artículo en la base de datos debe almacenar de manera obligatoria las propiedades de: Nombre, Descripción, Precio, Stock e Imagen (ya sea mediante carga local o referencia a URL externa).

3.3 Carrito de Compras y Flujo de Compra

- **RF-3.1:** El cliente autenticado podrá añadir productos a su orden temporal desde el listado del catálogo o desde el detalle individual del producto.
- **RF-3.2:** La vista del carrito de compras debe proveer controles interactivos claros para: remover un producto por completo, incrementar la cantidad deseada o reducirla.
- **RF-3.3:** El sistema calculará en tiempo real el subtotal por fila (***Precio × Cantidad***) y el coste total agregado de la transacción, reflejándose matemáticamente exacto en la interfaz.
- **RF-3.4:** Al accionar el botón de "Confirmar Compra", la aplicación ejecutará atómicamente el siguiente flujo:
 - Registrar la cabecera del pedido (Orden) enlazada directamente al ID del usuario autenticado.
 - Generar las líneas de detalle (Items de la Orden) con los snapshots de precios y cantidades al momento de la compra.
 - Vaciar de forma inmediata los elementos almacenados en el carrito activo del usuario.

3.4 Vistas y Navegación (Frontend)

- **RF-4.1:** Maquetación web adaptativa, uniforme y con un diseño estético profesional implementado mediante el uso consistente de un framework de estilos (como **Bootstrap**).
- **RF-4.2:** La barra de navegación principal (Navbar) debe mutar de forma dinámica según el contexto de la sesión:
 - *Invitado:* Enlaces a Catálogo y Login.
 - *Cliente:* Enlaces a Catálogo, Carrito (con indicador dinámico de cantidad total de ítems) y Logout.
 - *Administrador:* Enlaces a Catálogo, Panel de Administración de Productos y Logout.

3.5 Validaciones y Mensajes de Feedback

- **RF-5.1:** Validación obligatoria a nivel de formulario y backend que restrinja el ingreso de precios o cantidades menores o iguales a cero (≤ 0).
- **RF-5.2:** Integración total con el **Django Messages Framework** para emitir alertas instantáneas ante acciones clave (ej: *"Producto añadido con éxito"*, *"Orden de compra procesada correctamente"*, *"Error: El stock solicitado no está disponible"*).

4. Requisitos No Funcionales y Arquitectura

- **Base de Datos:** Motor relacional ligero **SQLite3** unificado tanto para las fases de desarrollo local como en el entorno productivo final.
- **Servidor de Producción:** Orquestación de peticiones HTTP en producción a través del servidor de interfaz de pasarela de servidor web (WSGI) **gunicorn**.
- **Manejo de Estáticos:** Integración y optimización de la entrega de archivos CSS, JavaScript y multimedia mediante el middleware comprimido de **WhiteNoise**.
- **Seguridad Operacional:** Abstracción rigurosa de parámetros críticos del sistema (**SECRET_KEY** , estado de **DEBUG**) inyectados dinámicamente mediante variables de entorno en el host.

⚠ **NOTA CRÍTICA DE PERSISTENCIA EN RENDER (SQLite3):**

Los servicios Web estándar en Render poseen un almacenamiento efímero: toda información creada en caliente o el archivo **db.sqlite3** nativo se destruirán irreversiblemente cada vez que el contenedor sea reiniciado, actualizado o redespigado. Para mitigar esta condición, **es estrictamente obligatorio crear y asociar un Disco Persistente (Volume)** en el dashboard de Render e instruir a Django a operar la base de datos dentro del punto de montaje asignado.

5. Estrategia de Despliegue en Render (Configuración Manual)

Paso 1: Adaptación de Ajustes en el Proyecto (**settings.py**)

Asegúrate de incluir las dependencias requeridas en tu entorno de producción local y congelarlas en tu archivo de texto:

```
pip install gunicorn whitenoise
pip freeze > requirements.txt
```

Posteriormente, configura la lógica dinámica en el archivo **settings.py** para gestionar el almacenamiento persistente del archivo SQLite3 dependiendo del entorno detectado:

```

import os
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent

# Configuración de Seguridad mediante variables de entorno externas
SECRET_KEY = os.environ.get('SECRET_KEY', 'django-insecure-desarrollo-local-de-respaldo')
DEBUG = os.environ.get('DEBUG', 'True') == 'True'

ALLOWED_HOSTS = ['*'] # En producción, se recomienda especificar la URL provista por
Render

# Inyección de WhiteNoise para la distribución óptima de estáticos
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'whitenoise.middleware.WhiteNoiseMiddleware', # Posicionar inmediatamente debajo de
SecurityMiddleware
    # ... resto de componentes middleware de Django
]

# Enrutamiento de la Base de Datos SQLite3 según el entorno operativo
if not DEBUG:
    # Render monta el volumen persistente asignado en el directorio raíz especificado
    (ej: /data)
    DATABASE_DIR = '/data'
    if not os.path.exists(DATABASE_DIR):
        os.makedirs(DATABASE_DIR)
    DB_PATH = os.path.join(DATABASE_DIR, 'db.sqlite3')
else:
    DB_PATH = BASE_DIR / 'db.sqlite3'

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': DB_PATH,
    }
}

# Configuración y compresión de archivos estáticos en producción
STATIC_URL = 'static/'
STATIC_ROOT = BASE_DIR / 'staticfiles'
STORAGES = {
    "staticfiles": {
        "BACKEND": "whitenoise.storage.CompressedManifestStaticFilesStorage",
    },
}

```

Paso 2: Aprovisionamiento del Servicio Web en el Dashboard

- Inicia sesión en la plataforma de Render, despliega el menú de acciones y selecciona **New + → Web Service**.
- Vincula tu repositorio correspondiente de GitHub.
- Declara manualmente las siguientes configuraciones dentro del asistente web de aprovisionamiento:
 - **Name:** `tu-nombre-de-ecommerce`
 - **Language:** `Python`
 - **Branch:** `main` (o tu rama principal activa)
 - **Build Command:** `pip install -r requirements.txt && python manage.py collectstatic --no-input && python manage.py migrate`
 - **Start Command:** `gunicorn nombre_de_tu_proyecto.wsgi:application`

Paso 3: Creación y Anclaje del Disco Persistente (Volume)

- Una vez que el servicio web haya inicializado su pipeline, accede a la pestaña de administración lateral y selecciona la opción de **Disks**.
- Haz clic sobre el control interactivo **Add Disk**.
- Completa estrictamente el formulario web con los siguientes parámetros exactos:
 - **Name:** `sqlite-data`
 - **Mount Path:** `/data` (Esta ruta absoluta debe coincidir carácter por carácter con la variable `DATABASE_DIR` definida en tu `settings.py`)
 - **Size:** `1 GB` (Dimensión de almacenamiento ideal dentro del nivel inicial, idónea para resguardar miles de transacciones y registros).

Paso 4: Inyección Manual de Variables de Entorno

- Desde la barra de utilidades del servicio en Render, navega hacia la sección de **Environment**.
- Añade de forma explícita los siguientes pares de clave y valor:
 - `DEBUG = False`
 - `SECRET_KEY = [IntroduceUnaCadenaAlfaNumericaComplejaYExtensaParaProduccion]`

6. Criterios de Aceptación (Definición de "Terminado")

- Un usuario autenticado puede añadir múltiples artículos al carrito de compras, alterar cantidades, remover un registro y verificar que los subtotales e importes generales se calculen matemáticamente sin discrepancias.
- Al accionar el control "Confirmar Compra", los registros correspondientes impactan de inmediato las tablas de Ordenes en la base de datos vinculadas al ID de sesión, y la interfaz vacía el carrito en su totalidad.

- Si un cliente sin privilegios elevados o un usuario anónimo intenta forzar la barra de direcciones del navegador hacia el endpoint de gestión de productos, el servidor responderá con una denegación explícita **403 Forbidden** o lo redirigirá al formulario de inicio de sesión.
- El despliegue en la infraestructura web concluye con éxito sin errores. Tras ejecutar un vaciado de caché y redespliegue de prueba (*Manual Deploy* → *Clear Cache & Deploy*), el catálogo precargado y los pedidos de prueba ****permanecen estables y sin pérdidas dentro de la aplicación****.